

Devboard Frontend — Docker & Multi-Stage Docker Notes

1. Project Structure

```
Devboard/
  Dockerfile
  Dockerfile.multistage
  package.json
  package-lock.json
  vite.config.js
  tailwind.config.js
  postcss.config.js
  index.html
  public/
  src/
  README.md
  .dockerignore
  .gitignore
```

Important Files

File	Purpose
<code>`package.json`</code>	Application dependencies and scripts
<code>`package-lock.json`</code>	Exact dependency versions
<code>`src/`</code>	React/Vite source code
<code>`public/`</code>	Static files
<code>`vite.config.js`</code>	Vite configuration
<code>`Dockerfile`</code>	Normal Docker image
<code>`Dockerfile.multistage`</code>	Multi-stage production-style image
<code>`.dockerignore`</code>	Files excluded from Docker build context

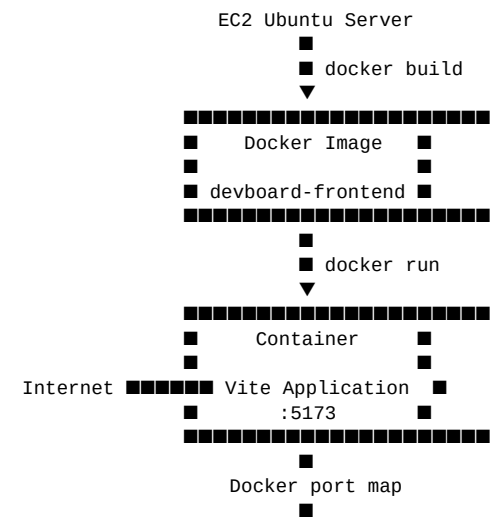
2. Two Docker Approaches

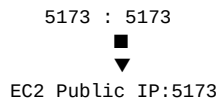
This lab demonstrates two approaches for the same Vite/React frontend:

- 1. **Normal Dockerfile** — the Vite development server runs inside the container.
- 2. **Multi-stage Dockerfile** — the application is built in a builder stage and only the required production build is copied into a smaller runtime image.

The main reason the first image became large was the combination of a full Node image, `node_modules`, source code, and development/build dependencies.

3. Overall Docker Concept





Example:

```
http://<EC2-PUBLIC-IP>:5173
```

EXPOSE does not open an AWS port. Docker port publishing and an appropriate AWS Security Group rule are still required.

4. Install Docker

```
sudo apt-get update
sudo apt-get install docker.io
docker -v
```

Check Docker service:

```
sudo systemctl status docker
```

Allow the Ubuntu User to Run Docker

```
sudo usermod -aG docker $USER
newgrp docker
```

Test:

```
docker ps
```

5. Basic Docker Commands

Show running containers:

```
docker ps
```

Show all containers, including stopped containers:

```
docker ps -a
```

Show local images:

```
docker images
```

Show Docker information:

```
docker info
```

6. Normal Dockerfile

A corrected development Dockerfile:

```
FROM node:22

WORKDIR /app

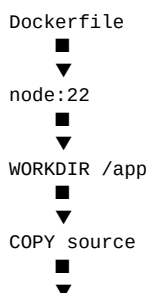
COPY . .

RUN npm install

EXPOSE 5173

CMD ["npm", "run", "dev", "--", "--host", "0.0.0.0", "--port", "5173"]
```

Dockerfile Flow



```
npm install
  ▼
node_modules
  ▼
Vite development server
  ▼
Port 5173
```

7. Dockerfile Instructions

`FROM`

```
FROM node:22
```

Provides the Node.js base image.

`WORKDIR`

```
WORKDIR /app
```

Sets /app as the working directory inside the image.

`COPY`

```
COPY . .
```

Copies the Docker build context into /app.

`RUN`

```
RUN npm install
```

Installs dependencies while the image is being built.

`EXPOSE`

```
EXPOSE 5173
```

Documents that the application listens on port 5173. It does not publish the port by itself.

`CMD`

```
CMD ["npm", "run", "dev", "--", "--host", "0.0.0.0", "--port", "5173"]
```

Starts the Vite development server when the container starts.

8. `.dockerignore`

Recommended:

```
node_modules
dist
.git
.gitignore
Dockerfile
Dockerfile.multistage
README.md
npm-debug.log
```

Why use `.dockerignore`?

- Reduces Docker build context.
- Prevents unnecessary files from being copied.
- Avoids copying local `node_modules`.
- Avoids copying `.git` history.
- Can improve build performance.

9. Important `COPY` Syntax

Correct:

```
COPY . .
```

The first . is the source and the second . is the destination.

Do not write:

```
COPY.
```

10. First Docker Build

```
docker build -t devboard-frontend .
```

Meaning:



Result:

```
devboard-frontend:latest
```

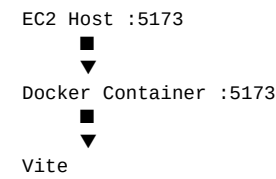
11. Run the Container

```
docker run -d \
-p 5173:5173 \
--name devboard-frontend-container \
devboard-frontend
```

Meaning:

Option	Meaning
-d	Detached/background mode
-p 5173:5173	Host port 5173 → container port 5173
--name	Assign a container name
devboard-frontend	Image to run

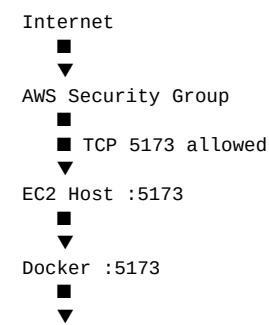
Port mapping:



12. AWS Security Group

For external browser access, the AWS Security Group must allow the required inbound TCP port.

Example:



Vite

Then access:

`http://<EC2-PUBLIC-IP>:5173`

13. Troubleshooting the First CMD Error

An incorrect command similar to:

```
CMD ["npm", "run", "-", "-host", "0.0.0.0", "--port", "5173"]
```

can cause:

```
npm error Missing script: "-host"
```

Why?

`npm run` expects a script name. The application script is normally:

```
npm run dev
```

Correct Docker command:

```
CMD ["npm", "run", "dev", "--", "--host", "0.0.0.0", "--port", "5173"]
```

The `--` separates npm arguments from Vite arguments.

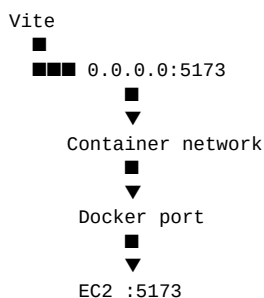
14. Why `0.0.0.0`?

Inside a container, binding only to localhost can make the application inaccessible through the published Docker port.

Use:

```
0.0.0.0
```

so Vite listens on the container's network interfaces.



15. Check the Container

```
docker ps
docker ps -a
```

Logs:

```
docker logs devboard-frontend-container
```

Enter the container:

```
docker exec -it devboard-frontend-container sh
```

If Bash exists:

```
docker exec -it devboard-frontend-container bash
```

Alpine-based images commonly use `sh` rather than `bash`.

16. `curl` Missing in Alpine

If this fails inside an Alpine container:

```
curl http://localhost:4173
```

with:

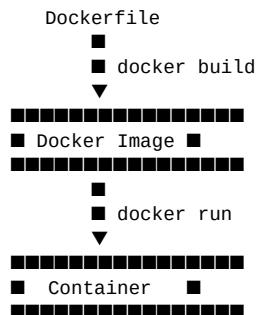
```
curl: not found
```

you can install it temporarily:

```
apk add --no-cache curl
```

For production images, avoid adding unnecessary troubleshooting tools unless they are actually required.

17. Image vs Container



Image

A packaged template used to create containers.

Container

A running or stopped instance created from an image.

18. Why the Normal Image Became Large

Your lab showed approximately:

```
Normal image: ~2.24 GB
Node base image: ~1.64 GB
```

Exact Docker size accounting depends on layers and local storage.

Main contributors:

```
node:22
+
application source
+
node_modules
+
development dependencies
+
build tools
+
Docker layers
↓
Large image
```

`npm install` may install hundreds of packages and their dependencies.

19. Memory and Swap

On a small EC2 instance, Docker builds and `npm` operations can create memory pressure.

You created a 2 GB swap file:

```
sudo fallocate -l 2G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile
```

Check:

```
free -h
```

Swap provides disk-backed virtual memory. It is much slower than RAM and should not be treated as a replacement for RAM.

20. RAM vs Disk

RAM and disk are different resources.

Check RAM and swap:

```
free -h
```

Check disk storage:

```
df -h
```

Check Docker image storage:

```
docker images
```

Docker disk usage:

```
docker system df
```

21. Multi-Stage Dockerfile

Example lab Dockerfile:

```
# Stage 1: Builder
FROM node:22 AS builder

WORKDIR /app

COPY . .

RUN npm install

RUN npm run build


# Stage 2: Runtime
FROM node:24-alpine AS runner

WORKDIR /app

COPY package.json .

RUN npm install -g vite

COPY --from=builder /app/dist ./dist

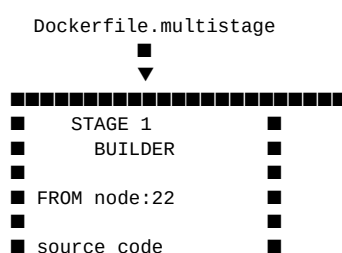
EXPOSE 4173

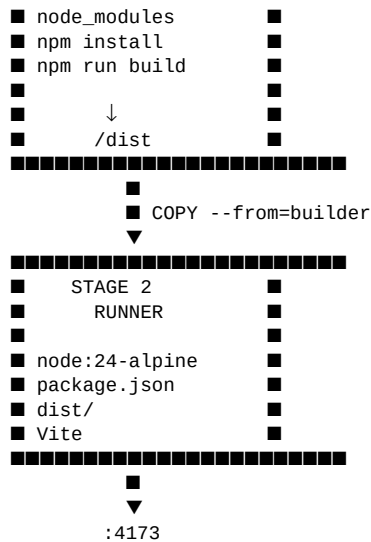
CMD ["vite", "preview", "--host", "0.0.0.0", "--port", "4173"]
```

The important concept is:

```
Builder Stage
■
■ npm install
■ npm run build
▼
dist/
■
■ COPY --from=builder
▼
Runtime Stage
■
▼
Small final image
```

22. Multi-Stage Build Diagram





23. What Is `dist/`?

Vite converts the source application into production build files:

```
npm run build
```

Example:

```

src/
├──
└──
  npm run build
  ├──
  └──
    dist/
    ├── index.html
    ├── assets/
    │   ├── index-xxxx.js
    │   └── index-xxxx.css
  
```

The `dist/` directory contains the built frontend assets.

24. `COPY --from=builder`

```
COPY --from=builder /app/dist ./dist
```

This means:

```
Builder stage:
/app/dist
```

```

├──
├── copy only dist
└──
  
```

```
Runtime stage:
/app/dist
```

It does not copy the entire builder filesystem.

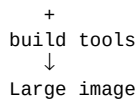
This is the core idea behind multi-stage Docker builds.

25. Why Multi-Stage Images Are Smaller

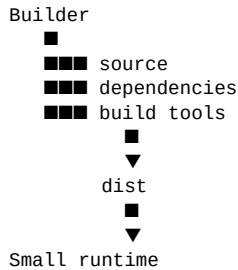
Normal approach:

```

node:22
+
source
+
node_modules
+
development dependencies
  
```

Multi-stage approach:



Your lab showed approximately:

Normal image:
~2.24 GB

Multi-stage image:
~387 MB

The exact size varies with the application, base images, layers, and Docker storage accounting.

26. Why `node:24-alpine`?

The runtime stage uses:

```
FROM node:24-alpine AS runner
```

Alpine-based images are designed to be lightweight.

So the final runtime can contain much less than the full builder environment.

27. Normal vs Multi-Stage

Feature	Normal Dockerfile	Multi-stage Dockerfile
Build stages	One	Multiple
Source in final image	Usually	Can be excluded
Build dependencies	Usually remain	Stay in builder
Image size	Usually larger	Usually smaller
Complexity	Simple	Slightly more complex
Development	Convenient	Not usually the main goal
Production	Possible	Common pattern
Runtime contents	More packages	Can be minimal
Security surface	Potentially larger	Can be reduced

28. `npm install` vs `npm ci`

For Docker/CI builds, when package-lock.json is committed, a common approach is:

```
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build
```

npm ci installs according to the lock file and is designed for clean, repeatable CI-style installs.

Example improved builder:

```
FROM node:22 AS builder

WORKDIR /app
```

```

COPY package*.json ./

RUN npm ci

COPY . .

RUN npm run build

```

29. Production-Style Vite Architecture

For a Vite frontend, a common production pattern is to use Nginx as the final static-file server.

```

# Stage 1: Build
FROM node:22 AS builder

WORKDIR /app

COPY package*.json ./

RUN npm ci

COPY . .

RUN npm run build

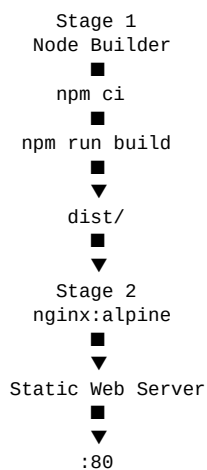

# Stage 2: Runtime
FROM nginx:alpine

COPY --from=builder /app/dist /usr/share/nginx/html

EXPOSE 80

```

Architecture:



For a production deployment, Nginx or another static web server is commonly used rather than relying on `vite preview`. `vite preview` is primarily useful for previewing the production build.

30. Docker Cleanup Commands

Remove a stopped container:

```
docker rm devboard-frontend-container
```

Force remove:

```
docker rm -f devboard-frontend-container
```

Remove an image:

```
docker rmi devboard-frontend:latest
```

Remove unused builder cache:

```
docker builder prune
```

More aggressive cache cleanup:

```
docker builder prune -a -f
```

Remove unused images:

```
docker image prune
```

More aggressive:

```
docker image prune -a
```

Remove unused Docker objects:

```
docker system prune
```

More aggressive:

```
docker system prune -a
```

Use aggressive cleanup commands carefully.

31. Multi-Stage Build Commands

Build:

```
docker build \
  -f Dockerfile.multistage \
  -t devboard-frontend-multi-stage .
```

Run:

```
docker run -d \
  -p 4173:4173 \
  --name devboard-frontend-multi-stage-container \
  devboard-frontend-multi-stage
```

Check:

```
docker ps
docker logs devboard-frontend-multi-stage-container
```

Enter:

```
docker exec -it devboard-frontend-multi-stage-container sh
```

32. Docker Storage Investigation

Useful commands:

```
docker system df
docker images
docker ps -a
docker volume ls
docker network ls
docker builder prune
```

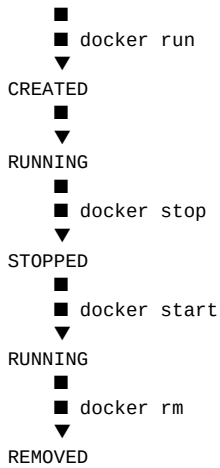
Docker storage can contain:

```
Docker Storage
■
■■■ Images
■   ■■■ node:22
■   ■■■ node:24-alpine
■   ■■■ devboard-frontend
■   ■■■ devboard-frontend-multi-stage
■
■■■ Containers
■   ■■■ running
■   ■■■ stopped
■
■■■ Volumes
■■■ Networks
■■■ Build Cache
```

Removing a container does not automatically remove its image.

33. Container Lifecycle

```
docker build
  ■
  ▼
  IMAGE
```



Useful commands:

```
docker start <container>
docker stop <container>
docker restart <container>
docker rm <container>
docker rm -f <container>
```

34. Complete Lab Command Sheet

Install Docker

```
sudo apt-get update
sudo apt-get install docker.io
docker -v
sudo systemctl status docker
```

User permissions

```
sudo usermod -aG docker $USER
newgrp docker
```

Check Docker

```
docker ps
docker ps -a
docker images
docker info
```

Normal build

```
docker build -t devboard-frontend .
```

Normal run

```
docker run -d \
-p 5173:5173 \
--name devboard-frontend-container \
devboard-frontend
```

Logs

```
docker logs devboard-frontend-container
```

Shell

```
docker exec -it devboard-frontend-container sh
```

Multi-stage build

```
docker build \
-f Dockerfile.multistage \
-t devboard-frontend-multi-stage .
```

Multi-stage run

```
docker run -d \
-p 4173:4173 \
--name devboard-frontend-multi-stage-container \
devboard-frontend-multi-stage
```

Multi-stage logs

```
docker logs devboard-frontend-multi-stage-container
```

Multi-stage shell

```
docker exec -it devboard-frontend-multi-stage-container sh
```

35. Interview Questions and Answers

What is the difference between a Docker image and container?

Answer: An image is a packaged, immutable template. A container is a runtime instance created from that image.

Why was the first image large?

Answer: It used a full Node image and contained source code, node_modules, development dependencies, and build tools.

Why use a multi-stage Dockerfile?

Answer: Build dependencies can remain in the builder stage while only the required build output is copied into the final runtime image.

Why use .dockerignore?

Answer: It prevents unnecessary files such as .git, node_modules, and local build artifacts from being sent to the Docker build context.

Why use 0.0.0.0?

Answer: It allows the application to listen on the container's network interfaces so Docker port publishing can expose it externally.

What does -p 5173:5173 mean?

Answer: The first port is the host port and the second is the container port.

Does EXPOSE 5173 open an AWS port?

Answer: No. EXPOSE documents the container port. Docker port publishing and the AWS Security Group determine external connectivity.

Why use npm ci in Docker builds?

Answer: When a lock file is committed, npm ci provides a clean, lock-file-based dependency installation suitable for CI/build environments.

36. Final Mental Model

Remember these five concepts:

1. Dockerfile
↓
Instructions
2. docker build
↓
Image
3. docker run
↓
Container
4. -p HOST:CONTAINER
↓
Port mapping
5. Multi-stage
↓
Build → dist → small runtime

Normal Docker

```
node:22
+
source
+
node_modules
+
```

Vite
↓
npm run dev
↓
:5173

Multi-Stage Docker

node:22 builder
↓
npm ci
↓
npm run build
↓
dist/
↓
node:alpine runner
↓
vite preview
↓
:4173

Production-style Frontend

Node Builder
■
■ npm run build
▼
dist/
■
■ COPY --from=builder
▼
nginx:alpine
■
▼
Static React/Vite files
■
▼
Port 80

What I Practiced

- Installed Docker on Ubuntu EC2.
- Added the Ubuntu user to the Docker group.
- Built a Vite/React frontend Docker image.
- Ran the application inside a Docker container.
- Published container port 5173.
- Debugged a wrong CMD instruction.
- Used `docker ps`, `docker ps -a`, and `docker logs`.
- Entered containers using `docker exec`.
- Worked with Alpine's `sh`.
- Investigated Docker image size.
- Created and enabled a 2 GB swap file.
- Built a multi-stage Docker image.
- Used `COPY --from=builder`.
- Compared a large development image with a smaller runtime image.
- Learned why build dependencies do not need to be included in the final runtime image.
- Learned the difference between Docker EXPOSE, Docker port publishing, and AWS Security Group access.
- Learned a production-style Vite build using Nginx.